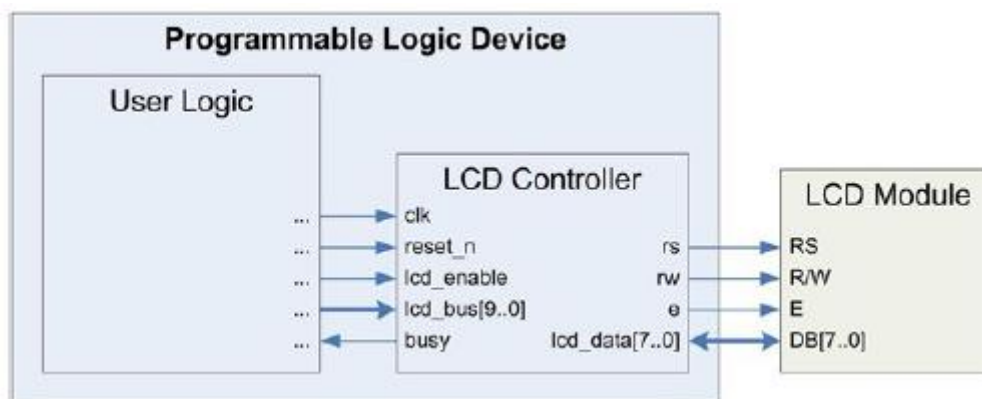


Introdução

O controlador gerencia a inicialização e o fluxo de dados para o HD44780 compatível com módulos LCD com interface de 8-bits.

Diagrama em Blocos do Circuitos



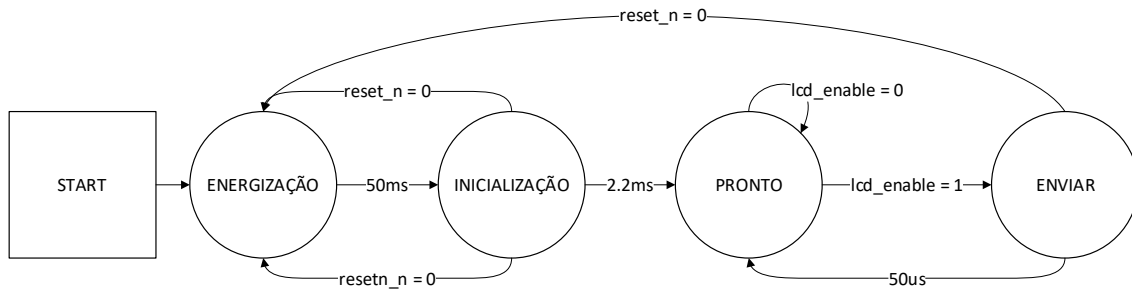
Descrição dos Pinos do Controlador

Nome	Bits	Modo	Descrição
clk	1	IN	Clock para o controlador LCD (50MHz).
reset_n	1	IN	Reset síncrono ativo baixo.
lcd_enable	1	IN	1: inicia transação com LCD. 0: não realiza transação.
lcd_bus	10	IN	Dados/instruções para o LCD (RS, RW, 1Byte dados).
busy	1	OUT	Feedback. 1. Controlador Ocupado. 2. Disponível.
rs	1	OUT	1: Envia dados. 0: Envia instruções
rw	1	OUT	1: Leitura do módulo LCD. 0: Escrita no módulo LCD.
e	1	OUT	1: habilita o módulo LCD. 0: desabilita.
lcd_data	8	INOUT	Barramento de dados entre controlador e o módulo.

Máquina de Estados

A FSM do controlador LCD consiste de cinco estados. Uma vez inicializado, imediatamente entra no estado de ENERGIZAÇÃO, onde aguarda 50ms garantindo uma tensão de alimentação estável. Então prossegue para o estado de INICIALIZAÇÃO. O controlador passa pela sequência de inicialização do módulo, que consistem na configuração dos parâmetros para valores pré-definidos no hardware. Este processo toma aproximadamente 2.2ms, em seguida o controlador passa para o estado PRONTO. Ele espera nesse estado até que a entrada lcd_enable seja ativada e, em seguida, avança para o estado ENVIAR. Então comunica os dados ao módulo LCD, conforme definido pela entrada lcd_bus. Após 50us, o controlador retorna ao estado PRONTO até novo aviso. Se um nível lógico baixo for aplicado à entrada reset_n, o controlador reinicializa para o estado ENERGIZAÇÃO e reinicializa.

Diagrama de Transição de Estados



Inicialização

O controlador LCD executa uma sequência de inicialização cada vez que é ligado (energizado) ou reinicializado através do pino reset_n. O controlador ativa o pino busy durante a inicialização. Uma vez que a inicialização é concluída, o pino busy é desativado e o controlador aguarda no estado PRONTO por uma entrada do bloco User Logic.

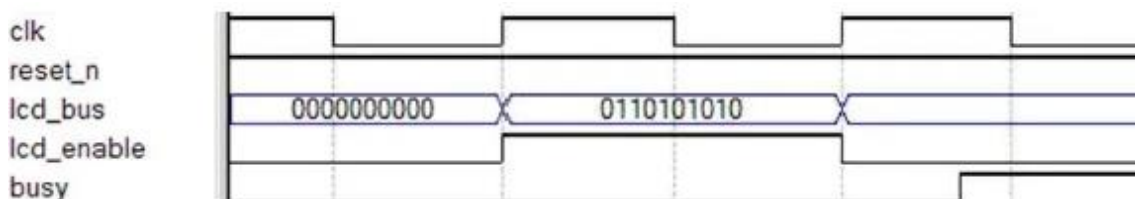
A sequência de inicialização especifica vários parâmetros LCD através de uma série de quatro comandos:

- Function Set : configura o display para 4-bit/8-bit, número de linhas do display, e a fonte dos caracteres.
- Display On/Off Control: liga/desliga o display, o cursor, e o piscar do cursor.
- Display Clear: limpa os dados do display.
- Entry Mode Set: configura modo incremental/decremental, e habilita/desabilita o deslocamento)

Transações

Após a desativação do pino busy, o controlador LCD entra no estado PRONTO. O bloco User Logic pode interagir através dos pinos lcd_enable e lcd_bus para realizar transações com o módulo LCD. O bloco User Logic inicia este processo emitindo os dados/instruções desejados para o lcd_bus e setando o pino lcd_enable. O controlador LCD então seta o pino busy e gerencia a transação. Ao terminar, o controlador desabilita o pino busy, indicando que está pronto para outra instrução.

Diagrama de Temporização das Transações



lcd_controller.vhd (LCD Controller)

```
1  LIBRARY IEEE;
2  USE IEEE.std_logic_1164.ALL;
3  ENTITY lcd_controller IS
4  GENERIC(
5      clk_freq      : INTEGER      := 50;    --clock principal em MHz
6      display_lines : STD_LOGIC   := '0';    --número de linhas do display (0 = uma linha, 1 = duas linhas)
7      character_font : STD_LOGIC   := '0';    --fonte (0 = 5x8 pontos, 1 = 5x10 pontos)
8      display_on_off : STD_LOGIC   := '1';    --display on/off (0 = off, 1 = on)
9      cursor        : STD_LOGIC   := '0';    --cursor on/off (0 = off, 1 = on)
10     blink          : STD_LOGIC   := '0';    --blink on/off (0 = off, 1 = on)
11     inc_dec        : STD_LOGIC   := '1';    --incremento/decremento (0 = decremento, 1 = incremento)
12     shift          : STD_LOGIC   := '0';    --shift on/off (0 = off, 1 = on)
13 )
14 PORT(
15     clk      : IN  STD_LOGIC; --clock principal
16     reset_n  : IN  STD_LOGIC; --ativo-baixo, reinicializa o LCD
17     lcd_enable : IN  STD_LOGIC; --retém os dados no controlador LCD
18     lcd_bus   : IN  STD_LOGIC_VECTOR(9 DOWNTO 0); --(9) rs (8) rw (7..0) dado char
19     busy      : OUT STD_LOGIC := '1'; --feedback do controlador (1)ocupado/(0)disponível
20     rw, rs, e : OUT STD_LOGIC; --leitura/escrita, instrução/dado, habilita LCD ativo-alto
21     lcd_data  : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)); --sinal de dado (char) para o LCD
22 END lcd_controller;
```

```
22 ARCHITECTURE bhv OF lcd_controller IS
23     --Declaração de estados da FSM
24     TYPE ESTADOS IS(ENERGIZACAO, INICIALIZACAO, PRONTO, ENVIAR);
25     SIGNAL estado : ESTADOS;
26 BEGIN
27     --FSM do
28     PROCESS(clk)
29         VARIABLE clk_count : INTEGER := 0; --contador para temporização de eventos
30     BEGIN
31         IF rising_edge(clk) THEN
32             --reinicializa a FSM
33             IF(reset_n = '0') THEN
34                 estado <= ENERGIZACAO;
35             END IF;
36             CASE estado IS
37                 -- espera 50ms para garantir a energização do LCD
38                 WHEN ENERGIZACAO =>
39                     busy <= '1';
40                     IF(clk_count < (50000 * clk_freq)) THEN --espera 50 ms
41                         clk_count := clk_count + 1;
42                     END IF;
```

```
43                     ELSE --energização completa
44                         clk_count := 0;
45                         rs <= '0';
46                         rw <= '0';
47                         lcd_data <= "00110000"; --8-bits 1L*16 5*8 function_set
48                         estado <= INICIALIZACAO;
49                     END IF;
50                 --sequência de inicialização do display LCD
51                 WHEN INICIALIZACAO =>
52                     busy <= '1'; --LCD ocupado
53                     clk_count := clk_count + 1;
54                     IF(clk_count < (10 * clk_freq)) THEN --function set
55                         lcd_data <= "0011" & display_lines & character_font & "00";
56                         e <= '1'; --habilita o LCD (executa o comando)
57                         estado <= INICIALIZACAO;
58                     ELSIF(clk_count < (60 * clk_freq)) THEN --espera 50 us
59                         lcd_data <= "00000000"; --nenhuma nova instrução, apenas aguarda
60                         e <= '0'; --desabilita o LCD
61                         estado <= INICIALIZACAO;
62                     ELSIF(clk_count < (70 * clk_freq)) THEN --display on/off control
63                         lcd_data <= "00001" & display_on_off & cursor & blink;
64                         e <= '1'; --habilita o LCD (executa o comando)
65                         estado <= INICIALIZACAO;
```

```

66 |         ELSIF(clk_count < (120 * clk_freq)) THEN --espera 50 us
67 |             lcd_data <= "00000000";
68 |             e <= '0';
69 |             estado <= INICIALIZACAO;
70 |         ELSIF(clk_count < (130 * clk_freq)) THEN --display clear
71 |             lcd_data <= "00000001";
72 |             e <= '1'; --habilita o LCD (executa o comando)
73 |             estado <= INICIALIZACAO;
74 |         ELSIF(clk_count < (2130 * clk_freq)) THEN --wait 2 ms
75 |             lcd_data <= "00000000";
76 |             e <= '0';
77 |             estado <= INICIALIZACAO;
78 |         ELSIF(clk_count < (2140 * clk_freq)) THEN --entry mode set
79 |             lcd_data <= "000001" & inc_dec & shift;
80 |             e <= '1'; --habilita o LCD (executa o comando)
81 |             estado <= INICIALIZACAO;
82 |         ELSIF(clk_count < (2200 * clk_freq)) THEN --wait 60 us
83 |             lcd_data <= "00000000";
84 |             e <= '0';
85 |             estado <= INICIALIZACAO;
86 |         ELSE --initialization complete
87 |             clk_count := 0;
88 |             busy <= '0';
89 |             estado <= PRONTO;
90 |         END IF;
91 |     --wait for the enable signal and then latch in the instruction

```

```

91 |     --wait for the enable signal and then latch in the instruction
92 |     WHEN PRONTO =>
93 |         IF(lcd_enable = '1') THEN
94 |             busy <= '1';
95 |             rs <= lcd_bus(9);
96 |             rw <= lcd_bus(8);
97 |             lcd_data <= lcd_bus(7 DOWNT0 0);
98 |             clk_count := 0;
99 |             estado <= ENVIAR;
100 |         ELSE
101 |             busy <= '0';
102 |             rs <= '0';
103 |             rw <= '0';
104 |             lcd_data <= "00000000";
105 |             clk_count := 0;
106 |             estado <= PRONTO;
107 |         END IF;

```

```

108 |     --envia instrução para o LCD
109 |     WHEN ENVIAR =>
110 |         busy <= '1'; --LCD ocupado
111 |         IF(clk_count < (50 * clk_freq)) THEN --espera 50 us
112 |             IF(clk_count < clk_freq) THEN --enable negativo
113 |                 e <= '0'; --desabilita o LCD
114 |             ELSIF(clk_count < (14 * clk_freq)) THEN --enable positivo em metade do ciclo (25us)
115 |                 e <= '1'; --habilita o LCD (executa o comando)
116 |             ELSIF(clk_count < (27 * clk_freq)) THEN --enable negativo na outra metade do ciclo (25us)
117 |                 e <= '0'; --desabilita o LCD
118 |             END IF;
119 |             clk_count := clk_count + 1;
120 |             estado <= ENVIAR;
121 |         ELSE
122 |             clk_count := 0;
123 |             estado <= PRONTO;
124 |         END IF;
125 |     END CASE;
126 | END IF;
127 | END PROCESS;
128 | END bhv;

```

lcd_logic.vhd (User Logic)

```
1  LIBRARY IEEE;
2  USE IEEE.std_logic_1164.all;
3  USE WORK.lcd_vhdl_package.ALL;
4  ENTITY lcd_logic IS
5  PORT( clk      : IN  STD_LOGIC; --clock principal
6        lcd_busy : IN  STD_LOGIC; --feedback do controlador (1)ocupado/(0)disponível
7        key       : IN  STD_LOGIC_VECTOR(4 DOWNTO 1); --botões
8        lcd_e      : OUT STD_LOGIC; --retem os dados no controlador LCD
9        lcd_bar    : OUT STD_LOGIC_VECTOR(9 DOWNTO 0) --(9) rs (8) rw (7..0) dado char
10 );
11 END lcd_logic;
```

```
12 ARCHITECTURE bhv OF lcd_logic IS
13     --Registradores
14     SIGNAL lcd_enable : STD_LOGIC;
15     SIGNAL lcd_bus    : STD_LOGIC_VECTOR(9 DOWNTO 0);
16     --Barramento de dados do display
17     SIGNAL L1 : std_logic_vector (127 DOWNTO 0);
18     SIGNAL L2 : std_logic_vector (127 DOWNTO 0);
19
20     --constantes
21     CONSTANT frase1_1 : std_logic_vector (127 DOWNTO 0) := to_std_logic_vector(" ELETRONICA ");--16 caracteres!!!
22     CONSTANT frase1_2 : std_logic_vector (127 DOWNTO 0) := to_std_logic_vector(" Digital UFPE ");--16 caracteres!!!
23     CONSTANT frase2_1 : std_logic_vector (127 DOWNTO 0) := to_std_logic_vector(" VHDL ");--16 caracteres!!!
24     CONSTANT frase2_2 : std_logic_vector (127 DOWNTO 0) := to_std_logic_vector("FPGA Cyclone IV ");--16 caracteres!!!
```

```
25 BEGIN
26     --atribuição contínua das saídas registradas
27     lcd_e <= lcd_enable;
28     lcd_bar <= lcd_bus;
29
30     --seleção do conteúdo através do key1
31     L1 <= frase1_1 WHEN (key(1) = '0') ELSE frase2_1;
32     L2 <= frase1_2 WHEN (key(1) = '0') ELSE frase2_2;
33
```

```
34     --Sequenciamento do envio de cada caractere de L1 e L2
35     PROCESS(clk)
36     VARIABLE char : INTEGER RANGE 0 TO 34 := 0; --6 bits
37     BEGIN
38         IF rising_edge(clk) THEN
39             IF (lcd_busy = '0' AND lcd_enable = '0') THEN
40                 lcd_enable <= '1'; --habilita o LCD
41                 IF (char < 34) THEN
42                     char := char + 1; --incrementa o estado
43                     ELSE char := 0; --reinicia o estado
44                 END IF;
45                 CASE char IS --verifica o estado atual
46                     WHEN 0 => lcd_bus <= "00" & "10000000"; --inst. linha 1
47                     WHEN 1 => lcd_bus <= "10" & L1(127 DOWNTO 120); --prim. char da linha 1
48                     WHEN 2 => lcd_bus <= "10" & L1(119 DOWNTO 112);
49                     WHEN 3 => lcd_bus <= "10" & L1(111 DOWNTO 104);
50                     WHEN 4 => lcd_bus <= "10" & L1(103 DOWNTO 96);
51                     WHEN 5 => lcd_bus <= "10" & L1(95 DOWNTO 88);
52                     WHEN 6 => lcd_bus <= "10" & L1(87 DOWNTO 80);
53                     WHEN 7 => lcd_bus <= "10" & L1(79 DOWNTO 72);
54                     WHEN 8 => lcd_bus <= "10" & L1(71 DOWNTO 64);
55                     WHEN 9 => lcd_bus <= "10" & L1(63 DOWNTO 56);
56                     WHEN 10 => lcd_bus <= "10" & L1(55 DOWNTO 48);
```

```

57      WHEN 11 => lcd_bus <= "10" & L1(47 DOWNT0 40);
58      WHEN 12 => lcd_bus <= "10" & L1(39 DOWNT0 32);
59      WHEN 13 => lcd_bus <= "10" & L1(31 DOWNT0 24);
60      WHEN 14 => lcd_bus <= "10" & L1(23 DOWNT0 16);
61      WHEN 15 => lcd_bus <= "10" & L1(15 DOWNT0 8);
62      WHEN 16 => lcd_bus <= "10" & L1(7 DOWNT0 0); --ult char da linha 1
63      WHEN 17 => lcd_bus <= "00" & "11000000"; --inst. linha 2
64      WHEN 18 => lcd_bus <= "10" & L2(127 DOWNT0 120); --prim. char da linha 2
65      WHEN 19 => lcd_bus <= "10" & L2(119 DOWNT0 112);
66      WHEN 20 => lcd_bus <= "10" & L2(111 DOWNT0 104);
67      WHEN 21 => lcd_bus <= "10" & L2(103 DOWNT0 96);
68      WHEN 22 => lcd_bus <= "10" & L2(95 DOWNT0 88);
69      WHEN 23 => lcd_bus <= "10" & L2(87 DOWNT0 80);
70      WHEN 24 => lcd_bus <= "10" & L2(79 DOWNT0 72);
71      WHEN 25 => lcd_bus <= "10" & L2(71 DOWNT0 64);
72      WHEN 26 => lcd_bus <= "10" & L2(63 DOWNT0 56);
73      WHEN 27 => lcd_bus <= "10" & L2(55 DOWNT0 48);
74      WHEN 28 => lcd_bus <= "10" & L2(47 DOWNT0 40);
75      WHEN 29 => lcd_bus <= "10" & L2(39 DOWNT0 32);
76      WHEN 30 => lcd_bus <= "10" & L2(31 DOWNT0 24);
77      WHEN 31 => lcd_bus <= "10" & L2(23 DOWNT0 16);
78      WHEN 32 => lcd_bus <= "10" & L2(15 DOWNT0 8);
79      WHEN 33 => lcd_bus <= "10" & L2(7 DOWNT0 0);--ult. char da linha 2
80      WHEN OTHERS => lcd_enable <= '0'; --desabilita o LCD
81  END CASE;

```

```

82  ELSE
83      lcd_enable <= '0'; --desabilita o LCD
84  END IF;
85  END IF;
86  END PROCESS;
87  END bhv;

```